

GENOMIC MUTATION TRACKER

Complete Project Guide

insert_db.py + SQL + Terminal Rules + Everything Explained

#	Topic
1	Are Syntax Questions Asked in Interviews?
2	Terminal Rules — How to Run Everything
3	insert_db.py — Complete Explanation
4	MySQL Connection Syntax Explained
5	CSV Reading Syntax Explained
6	SQL Changes in main.cpp
7	db.cpp and db.h Explained
8	patient.h — Why We Created It
9	All Files and Their Purpose
10	Complete Project Flow
11	Interview Answers

1. Are Syntax Questions Asked in Interviews?

Short answer: NO. Interviewers do not test if you memorized exact syntax.

What they actually test:

What They Test	Example Question
Understanding of WHAT the code does	"What does <code>mysql.connector.connect()</code> do?"
Understanding of WHY you wrote it	"Why did you use Python instead of C++ for MySQL?"
Logic and reasoning	"What happens if the connection fails?"
Debugging ability	"What would you check if insert failed?"
Trade-off awareness	"Why <code>unordered_map</code> over <code>map</code> for frequencies?"

Confusing syntax you do NOT need to memorize:

`mysql.connector.connect()`, `csv.DictReader()`, `chrono::high_resolution_clock::now()`, `priority_queue`, `RANK() OVER()` — for all of these, just understand WHAT they do and WHY you used them. You can always say 'I used Python's mysql connector library, the exact method name I'd look up in docs.' That is a perfectly acceptable answer.

Syntax you DO need to know cold:

Syntax	Why You Need It
<code>git add</code> , <code>commit</code> , <code>push</code> , <code>pull</code>	Asked in every interview
<code>SELECT</code> , <code>WHERE</code> , <code>GROUP BY</code> , <code>ORDER BY</code>	Basic SQL — always tested
<code>JOIN</code> types (<code>INNER</code> , <code>LEFT</code> , <code>RIGHT</code>)	Very commonly asked
<code>vector</code> , <code>map</code> , <code>unordered_map</code> basics	DSA interviews
Time complexity (O notation)	Always asked for algorithms
What is KMP? What is LPS?	You built it — must explain it

2. Terminal Rules — How to Run Everything

Rule 1: Always be in the right folder first

```
cd D:\Projects\genomic_tracker
```

Every command must be run from inside the genomic_tracker folder. If you open a new terminal, always cd here first.

Rule 2: On Windows use .\ not ./

```
.\tracker (Windows PowerShell - CORRECT) ./tracker (Linux/Mac - WRONG on Windows)
```

Rule 3: Add MySQL to PATH every new terminal session

```
$env:PATH += ";C:\Program Files\MySQL\MySQL Server 8.0\bin"
```

This only lasts for the current terminal session. If you open a new terminal, run this again. To make it permanent, add MySQL bin to Windows Environment Variables.

Rule 4: Compile order matters

```
# Generator (runs alone): g++ generator.cpp -o generator .\generator # Main tracker (needs  
all 4 files): g++ main.cpp kmp.cpp rabin_karp.cpp db.cpp -o tracker .\tracker # Python  
script (after tracker creates CSV files): python insert_db.py
```

Rule 5: If you change any .cpp file, recompile

Changing main.cpp and running the old tracker.exe does nothing. The .exe is a snapshot of when you last compiled. Always recompile after any code change.

Rule 6: Git always in 3 steps

```
git add . git commit -m "describe what you changed" git push
```

Complete run sequence from scratch:

Step	Command	What it does
1	cd D:\Projects\genomic_tracker	Go to project folder
2	g++ generator.cpp -o generator	Compile generator
3	.\generator	Create patients.txt
4	g++ main.cpp kmp.cpp rabin_karp.cpp db.cpp -o tracker .\tracker	Compile main project
5	.\tracker	Run KMP + Rabin-Karp + stats, creates CSV files
6	python insert_db.py	Insert CSV data into MySQL
7	Open MySQL Workbench	Run setup.sql queries to see analytics

3. insert_db.py — Complete Explanation

What this file does in one line:

Reads the 3 CSV files that C++ created and inserts their data into MySQL tables.

Why Python and not C++:

Your g++ compiler is 32-bit (MinGW) but MySQL 8.0 is 64-bit. They are architecturally incompatible at link time. Python's mysql-connector handles this internally. This is called polyglot architecture — using the right language for each job.

Line by line explanation:

IMPORTS

```
import mysql.connector import csv
```

mysql.connector is the Python library that talks to MySQL. csv is Python's built-in library for reading CSV files. Both need to be imported before use — like #include in C++.

CONNECTION

```
conn = mysql.connector.connect( host="localhost", user="root", password="your_password",  
database="genomic_tracker" ) cursor = conn.cursor()
```

Parameter	Value	Meaning
host	"localhost"	MySQL is on YOUR computer, not a remote server. localhost = this machine.
user	"root"	The MySQL admin username. root is the default superuser created during MySQL installation.
password	"your_password"	Whatever you set during MySQL installation. This is YOUR password, not a standard one.
database	"genomic_tracker"	Which database to connect to. We created this with CREATE DATABASE earlier.

conn vs cursor — what is the difference?

```
conn = the door to MySQL (the connection itself) cursor = the pen to write queries through  
that door Think of it like: conn = phone call connected to a bank cursor = the agent on the  
phone who executes your requests
```

CLEARING OLD DATA

```
cursor.execute("DELETE FROM mutation_results") cursor.execute("DELETE FROM patients")  
cursor.execute("DELETE FROM mutation_frequencies") conn.commit()
```

Why delete mutation_results BEFORE patients? Because of FOREIGN KEY constraint:

```
mutation_results.patient_id REFERENCES patients.patient_id If you delete patients first:  
MySQL error: "mutation_results still references these patients!" Always delete CHILD table  
first (mutation_results) Then delete PARENT table (patients) This respects the foreign key  
relationship.
```

conn.commit() after deletes is mandatory. Without commit, deletions are temporary and get rolled back when connection closes.

READING CSV AND INSERTING

```
with open("results.csv", "r") as f: reader = csv.DictReader(f) for row in reader:
    cursor.execute( INSERT INTO patients VALUES (%s,%s,%s,%s,%s,%s,%s,%s), (row["patient_id"],
    int(row["age"]), ...) ) conn.commit()
```

Breaking this down:

```
with open("results.csv", "r") as f: Opens results.csv for reading ("r" = read mode) "with"
automatically closes the file when done f is the file object csv.DictReader(f): Reads CSV
with first row as column names Each row becomes a dictionary: row = {"patient_id": "P001",
"age": "58", "total_mutations": "21", ...} for row in reader: Loops through every line in
the CSV Each iteration = one patient cursor.execute(query, values): Runs an SQL INSERT
statement %s are placeholders (like printf in C++) Values are passed as a tuple separately
This prevents SQL injection attacks int(row["age"]): CSV gives everything as strings: "58"
MySQL expects integer: 58 int() converts string to integer float() converts string to
decimal conn.commit(): Saves all inserts permanently to disk Without this, inserts exist in
memory only If connection closes before commit, all lost
```

What is SQL Injection? (bonus interview topic)

If you built the query like this: query = 'INSERT INTO patients VALUES (" + row['id'] + ")' — a hacker could put SQL code inside the CSV file and it would execute. Using %s placeholders prevents this because Python treats the values as pure data, never as SQL commands.

4. MySQL Connection Syntax — All Confusing Parts

cursor.execute() vs conn.commit()

`cursor.execute("DELETE FROM patients")` ← sends command to MySQL
`cursor.execute("INSERT INTO ...")` ← sends another command
`conn.commit()` ← SAVES all of them permanently
Without `commit()`: Commands sent but not saved Like typing in Word but not pressing Ctrl+S
Connection closes → everything gone
With `commit()`: All changes permanently written to disk
Data survives closing Workbench, restarting computer

Why %s and not actual values directly?

BAD (SQL injection risk): `cursor.execute("INSERT INTO patients VALUES ('" + patient_id + "')")`
GOOD (safe): `cursor.execute("INSERT INTO patients VALUES (%s)", (patient_id,))`
The %s approach: Python escapes special characters automatically
Hacker cannot inject SQL through the data
Standard practice in ALL database code

Why (patient_id,) with a comma for single value?

In Python: `(patient_id)` = just parentheses around a value, NOT a tuple
`(patient_id,)` = a tuple with one element (comma makes it a tuple)
`cursor.execute` needs a TUPLE even for one value
So always write `(value,)` not `(value)` for single values
For multiple values: `(val1, val2, val3)` – no trailing comma needed

cursor.close() and conn.close()

`cursor.close()` ← close the pen (cursor)
`conn.close()` ← close the door (connection)
Always close in this order: cursor first, then conn
Like hanging up the phone properly
Frees up MySQL connection slots
Without closing, MySQL might run out of available connections

5. SQL-Related Changes in main.cpp

The main changes in main.cpp that relate to the database pipeline:

Change 1: Added db.h include

```
#include "db.h"
```

This tells main.cpp that insertToMySQL() function exists in db.cpp. Without this, compiler doesn't know what insertToMySQL is.

Change 2: Added unordered_map for frequencies

```
unordered_map freqMap; computeFrequencies(patients, freqMap);
```

freqMap stores global mutation counts across ALL patients. This is what gets inserted into mutation_frequencies table in MySQL.

Change 3: computeFrequencies function

```
void computeFrequencies(vector& patients, unordered_map& freqMap) { for (auto& p : patients) { for (auto& entry : p.mutationMap) { freqMap[entry.first] += entry.second.size(); } } }
```

entry.first = mutation name ("GGT") entry.second = positions vector ([23, 145, 302])
entry.second.size() = how many times found (3) freqMap["GGT"] += 3 means GGT found 3 times in this patient After all 100 patients: freqMap["GGT"] = 1637 total

Change 4: insertToMySQL call at end of main

```
// at end of main(), before return 0: insertToMySQL(patients, freqMap);
```

This one line triggers db.cpp to write all 3 CSV files. Passes patients vector (all 100 patients with their results) and freqMap (global frequencies).

Change 5: computeStats function

```
void computeStats(Patient& p) { p.uniqueMutations = p.mutationMap.size(); p.mutationDensity = (double)p.totalMutations / p.dna.length() * 1000.0; p.riskScore = (p.totalMutations * 0.5) + (p.uniqueMutations * 1.0) + (p.mutationDensity * 0.3); }
```

Field	Formula	Meaning
uniqueMutations	mutationMap.size()	Number of keys in map = different mutation TYPES found
mutationDensity	(total / dna_length) x 1000	Mutations per 1000 characters. Normalizes for comparison.
riskScore	total*0.5 + unique*1.0 + density*0.3	Weighted risk. Unique types weighted highest (most dangerous).

6. db.cpp and db.h — Explained

db.h — The Header

```
#ifndef DB_H #define DB_H #include "patient.h" #include using namespace std; void insertToMySQL(vector& patients, unordered_map& freqMap); #endif
```

db.h just declares that insertToMySQL exists and what parameters it takes. It includes patient.h because the function uses the Patient struct. main.cpp includes this header to know the function exists.

db.cpp — Writing 3 CSV Files

Originally db.cpp was supposed to connect directly to MySQL. But MinGW 32-bit g++ is incompatible with MySQL 64-bit libraries. So we changed db.cpp to write CSV files instead, and use Python to insert them.

Writing results.csv:

```
ofstream pFile("results.csv"); pFile << "patient_id,age,dna_length,..." << "\n"; // header
for (auto& p : patients) { pFile << p.id << "," << p.age << "," << ... << "\n"; }
pFile.close();
```

ofstream = output file stream. Works exactly like cout but writes to a file. First line is the header (column names). Then one line per patient. Always close() at end.

Writing mutation_results.csv:

```
for (auto& p : patients) { // each patient for (auto& entry : p.mutationMap) { // each
mutation type for (int pos : entry.second) { // each position mFile << p.id << "," <<
entry.first << "," // mutation name << pos << "," // position number << "KMP" << "\n"; } } }
```

Three nested loops because: one patient has multiple mutation types, and each mutation type appears at multiple positions. This creates one row per occurrence.

Writing frequencies.csv:

```
for (auto& entry : freqMap) { fFile << entry.first << "," // mutation name << entry.second
<< "\n"; // total count }
```

Simplest loop. Just one row per mutation type with its global count across all patients.

7. patient.h — Why We Created It

The problem we had:

Before patient.h: Patient struct defined in main.cpp db.cpp also needs Patient struct Option A: Copy struct into db.cpp too PROBLEM: two copies, if you add a field you must change it in TWO places, easy to forget one Option B: Move struct to patient.h GOOD: defined ONCE main.cpp includes it db.cpp includes it db.h includes it Change it once -> everyone gets the update

This is called Single Source of Truth — a fundamental software engineering principle.

The Patient struct fields:

Field	Type	What it stores
id	string	"P001" — patient identifier
age	int	34 — patient age
dna	string	"ATCGGTAGCT..." — 1000 char DNA sequence
totalMutations	int	Total mutation occurrences found by KMP
mutationMap	map<string,vector<int>>	"GGT" -> [23, 145, 302] — KMP results
rkMutationMap	map<string,vector<int>>	Same structure — Rabin-Karp results
rkTotalMutations	int	Total count from Rabin-Karp (for comparison)
uniqueMutations	int	How many different mutation TYPES found
mutationDensity	double	Mutations per 1000 chars (e.g. 21.0)
riskScore	double	Computed risk score (e.g. 20.8)

8. All Files and Their Purpose

File	Language	Purpose	Talks to
generator.cpp	C++	Generates patients.txt with 100 simulated patients. Age-compatibles mutation injection. Run once to create patients.txt	patients.txt
patient.h	C++	Defines Patient struct. Single source of truth. Shared across everyone.cpp, db.cpp, db.h.	Everyone.cpp, db.cpp, db.h.
kmp.h	C++	Declares buildLPS() and KMPsearch() functions.	kmp.cpp
kmp.cpp	C++	Implements KMP algorithm. buildLPS builds failure function. Called by rabin_karp.cpp for all occurrences.	Called by rabin_karp.cpp
rabin_karp.h	C++	Declares RabinKarpMulti() function.	rabin_karp.cpp
rabin_karp.cpp	C++	Implements rolling hash search. Groups patterns by length. Called by main.cpp for unique length.	Called by main.cpp
db.h	C++	Declares insertToMySQL() function.	db.cpp
db.cpp	C++	Writes 3 CSV files: results.csv, mutation_results.csv, frequencies.csv. Creates CSV files	Creates CSV files
main.cpp	C++	Orchestrates everything. Loads patients, runs both algorithms, computes stats, timing, top-K, calls insert_db.py	insert_db.py
insert_db.py	Python	Reads 3 CSV files, connects to MySQL, clears old data, inserts everything into MySQL database	MySQL database
setup.sql	SQL	Creates database and 3 tables. Contains all 5 analytics queries	MySQL Workbench
patients.txt	Data	Generated patient data. 100 lines, each: P001,34,ATCGGRead by main.cpp	Read by main.cpp
results.csv	Data	Patient stats output. One row per patient with all computed fields	Read by insert_db.py
mutation_results.csv	Data	One row per mutation occurrence with position and algorithm	Read by insert_db.py
frequencies.csv	Data	Global mutation frequency totals across all patients.	Read by insert_db.py
.gitignore	Config	Tells git to ignore *.exe and *.o files. Prevents binary files on GitHub.	Git/GitHub.
README.md	Docs	Project description for GitHub. How to run, what it does, tech stack	GitHub viewers

9. Complete Project Flow

STEP 1: generator.cpp Creates 100 patients with random DNA Age determines how many mutations injected Writes to patients.txt STEP 2: main.cpp loads patients.txt Reads file line by line Splits each line by comma using stringstream Fills Patient struct: id, age, dna Stores in vector STEP 3: KMP search (for each patient x each mutation) buildLPS(pattern) creates failure array KMPsearch(dna, pattern) finds all positions Stores in patient.mutationMap Accumulates patient.totalMutations STEP 4: Rabin-Karp search (for each patient) RabinKarpMulti(dna, all_mutations) Groups patterns by length One pass per unique length using rolling hash Stores in patient.rkMutationMap Cross-validated against KMP results STEP 5: computeStats() for each patient uniqueMutations = mutationMap.size() mutationDensity = (total/length) x 1000 riskScore = weighted formula STEP 6: computeFrequencies() Aggregates counts across all 100 patients freqMap["GGT"] = 1637 (total across everyone) STEP 7: printTopK(freqMap, 5) Pushes all into priority_queue (max heap) Pops top 5 by frequency Prints ranking STEP 8: insertToMySQL(patients, freqMap) Writes results.csv (patient stats) Writes mutation_results.csv (positions) Writes frequencies.csv (global counts) STEP 9: python insert_db.py Connects to localhost MySQL Clears old data (child tables first) Reads each CSV → inserts into MySQL table Commits after each table STEP 10: MySQL Workbench queries Top 10 risk patients (ORDER BY risk_score DESC) Age group analysis (CASE + GROUP BY) Common mutations (mutation_frequencies table) Window functions (RANK() OVER, AVG() OVER) Density comparison (GROUP BY age_group)

10. SQL Queries Explained

Query 1: Top 10 Risk Patients

```
SELECT patient_id, age, risk_score FROM patients ORDER BY risk_score DESC LIMIT 10;
```

ORDER BY risk_score DESC = sort by risk_score from highest to lowest. LIMIT 10 = show only top 10. Simple but very useful for doctors.

Query 2: Age Group Analysis (CASE statement)

```
SELECT CASE WHEN age < 30 THEN '20-29' WHEN age < 50 THEN '30-49' WHEN age < 70 THEN '50-69' ELSE '70-80' END AS age_group, COUNT(*) as total_patients, AVG(total_mutations) as avg_mutations, AVG(risk_score) as avg_risk FROM patients GROUP BY age_group ORDER BY age_group;
```

CASE WHEN = SQL's if-else statement age < 30 THEN '20-29' means: if age is less than 30, label it '20-29' Each patient gets assigned to exactly one group ELSE catches everyone who doesn't match earlier conditions AS age_group = gives the CASE result a column name GROUP BY age_group = groups all patients in same age bucket together AVG(total_mutations) = average mutations within each group COUNT(*) = how many patients are in each group

Query 3: Most Common Mutations

```
SELECT mutation_name, total_count FROM mutation_frequencies ORDER BY total_count DESC;
```

Simplest query. Just reads the frequencies table we populated. Shows which mutation is most prevalent in the patient population.

Query 4: Window Function — RANK() OVER()

```
SELECT patient_id, age, risk_score, RANK() OVER (ORDER BY risk_score DESC) as risk_rank, AVG(risk_score) OVER () as overall_avg_risk FROM patients LIMIT 10;
```

This is the most advanced query. Window functions compute values ACROSS rows without collapsing them (unlike GROUP BY). RANK() OVER (ORDER BY risk_score DESC): Assigns rank 1 to highest risk_score Rank 2 to second highest, etc. OVER() = defines the window (set of rows to rank across) ORDER BY inside OVER = sort order for ranking Tied values get same rank (both P061 and P068 got rank 1) AVG(risk_score) OVER (): Computes average risk across ALL patients Empty OVER() = window is entire table Shows 25.45 for every row (same overall average) Lets you compare each patient's risk to overall average Key difference from GROUP BY: GROUP BY collapses rows (100 rows -> 4 age groups) Window functions keep all rows and ADD computed columns

Query 5: Age Group Density Comparison

```
SELECT CASE WHEN age < 30 THEN '20-29' ... END AS age_group, AVG(mutation_density) as avg_density, MAX(risk_score) as max_risk, MIN(risk_score) as min_risk FROM patients GROUP BY age_group ORDER BY avg_density DESC;
```

Same CASE pattern but adds MAX() and MIN() to show spread within each age group. Shows which age group has the widest variation in risk scores.

11. Interview Answers — Ready to Use

Q: Tell me about your project

A: I built a genomic mutation tracker that searches patient DNA sequences for disease-causing patterns. I implemented KMP and Rabin-Karp algorithms in C++, benchmarked them, computed patient risk scores, and stored results in MySQL for analytics using SQL window functions.

Q: Why two algorithms for the same task?

A: To demonstrate trade-offs. KMP is $O(N+M)$ and optimal for single pattern search. Rabin-Karp uses rolling hash and is better for multiple patterns simultaneously. On our 1000-char DNA, KMP won due to lower setup overhead. At genome scale with 50+ patterns, Rabin-Karp would win due to its $O(1)$ sliding window.

Q: What is LPS and why does KMP need it?

A: LPS is Longest Proper Prefix which is also Suffix. For a pattern like ABABC, $LPS = [0,0,1,2,0]$. When a mismatch occurs during search, LPS tells us how far back to jump in the pattern without losing already-matched characters. This ensures i (text pointer) never goes back, giving $O(N+M)$ complexity.

Q: Why Python for MySQL and not C++?

A: MinGW g++ on Windows is 32-bit but MySQL 8.0 is 64-bit. The library files are architecturally incompatible at link time — `mysqlclient.lib` reports 'unhandled machine type 0x8664'. Python's `mysql-connector` handles this internally. This is polyglot architecture — using the right tool for each job, which is standard in production systems.

Q: What SQL concepts did you use?

A: CASE statements for age group bucketing, GROUP BY for aggregation, ORDER BY for ranking, RANK() window function for patient risk ranking with tied values handled correctly, AVG() OVER() for computing overall averages without collapsing rows, FOREIGN KEY constraints for referential integrity, and LIMIT for top-K queries.

Q: What is the difference between GROUP BY and window functions?

A: GROUP BY collapses multiple rows into one summarized row per group — 100 patients become 4 age groups. Window functions compute across rows but keep all rows intact, just adding computed columns. RANK() OVER() ranks every patient while still showing all 100 rows. This lets you compare individual values to group statistics in the same query.

Q: Is your data realistic?

A: It is a simulation. Mutation injection rate correlates with age to model the biological reality that older patients accumulate more DNA copying errors. I documented this limitation in the README. Production would use real data from NCBI GenBank. The algorithms are identical regardless of data source — only the data loading changes.

Q: What is your risk score formula and why those weights?

A: $Risk = totalMutations \times 0.5 + uniqueMutations \times 1.0 + mutationDensity \times 0.3$. Unique mutation types get highest weight because diversity of mutations is more clinically dangerous than repeating the same mutation. Total count is weighted 0.5 because repetition matters but less so. Density gets 0.3 because spread-out mutations are less concerning than clustered ones.

Q: What data structures did you use and why?

A: vector for patient list — dynamic array, easy iteration. map> for mutation positions — sorted by mutation name, $O(\log n)$ lookup, clean output. unordered_map for global frequencies — $O(1)$ average lookup, order doesn't matter for counting. priority_queue for top-K — max heap, automatically gives highest frequency on top in $O(\log n)$ per push.

Q: How would you scale this to real genome data?

A: Real human genome is 3 billion characters. I would use memory-mapped files instead of loading entire DNA into RAM. For 1000 patients that is 3TB — impossible to hold in memory. I would stream the DNA in chunks, use BLAST instead of manual KMP for production accuracy, store data in a distributed database like Cassandra or HBase, and use parallel processing across patients.

12. Project Summary

Category	Details
Languages	C++ (algorithms), Python (database), SQL (analytics)
Algorithms	KMP with LPS array, Rabin-Karp with rolling hash
Data Structures	vector, map, unordered_map, priority_queue, struct
Database	MySQL with 3 tables, 5 analytics queries, window functions
Files	15 files total across C++, Python, SQL, data, config, docs
Key Concepts	String matching, hashing, heaps, file I/O, header files, polyglot architecture, SQL window functions
Time Complexity	KMP: $O(N+M)$ Rabin-Karp: $O(N+M)$ avg Top-K: $O(n \log k)$
Limitation	Simulated data, 32-bit/64-bit MySQL incompatibility on Windows
Production upgrade	Real NCBI data, 64-bit compiler, streaming for large genomes

You built a complete end-to-end system in one session. That is genuinely impressive.